

CLUSTERING STATES INTO DIFFERENT COVID-19 ZONES

Hoàng-Nguyên Vũ

1. Mô tả

Bài tập thực hành KMeans nhằm phân vùng bệnh Covid-19 trong bài toán thực tế về dịch bệnh Sar-Cov2 năm 2021 là một ứng dụng quan trọng trong phân tích dữ liệu y tế. Mục tiêu của bài tập là sử dụng thuật toán KMeans để phân nhóm các khu vực hoặc quốc gia dựa trên các chỉ số quan trọng như số ca nhiễm, số ca hồi phục và số ca tử vong. Qua việc phân cụm, ta có thể xác định các khu vực có đặc điểm dịch tễ học tương đồng, từ đó hỗ trợ các nhà quản lý y tế đưa ra quyết định hiệu quả hơn trong việc phân bổ nguồn lực và triển khai các biện pháp phòng chống dịch phù hợp. Đây là một ví dụ điển hình về cách sử dụng machine learning để giải quyết các vấn đề thực tiễn trong y tế cộng đồng.

2. Nội dung

Chúng ta sẽ sử dụng thuật toán K-Means, đây là một thuật toán phân cụm không giám sát. Thuật toán k-means là một thuật toán lặp đi lặp lại, cố gắng chia tập dữ liệu thành K cụm (nhóm con) riêng biệt không chồng chéo, trong đó mỗi điểm dữ liệu chỉ thuộc về một cụm. Thuật toán cố gắng làm cho các điểm dữ liệu trong cùng một cụm càng giống nhau càng tốt, đồng thời giữ cho các cụm càng xa nhau càng tốt. Nó gán các điểm dữ liệu vào một cụm sao cho tổng khoảng cách bình phương giữa các điểm dữ liệu và tâm cụm là nhỏ nhất.

Bước 1: Load dataset: Tải tại đây

```
1 import numpy as np
2 import pandas as pd
3 from sklearn.cluster import KMeans
4 from matplotlib import pyplot as plt
5
6 df_india = # Your coder here to load dataset#
7 print(df_india)
```

	State	Confirmed	Recovered	Deaths
0	Maharashtra	41642	11726	1454
1	Tamil Nadu	13967	6282	95
2	Gujarat	12910	5488	773
3	Delhi	11659	5567	194
4	Rajasthan	6227	3485	151
5	Madhya Pradesh	5981	2844	271

Bước 2: Thực hiện tính tỉ lệ hồi phục và tử vong cho mỗi khu vực:

Chúng ta sẽ thực hiện tính tỉ lệ hồi phục và tử vong từ tập dữ liệu trên theo công thức sau:

$$\text{Tỉ lệ hồi phục} = \frac{\text{Hồi phục}}{\text{Số ca xác nhận}} * 100\% \quad (1)$$

$$\text{Tỉ lệ tử vong} = \frac{\text{Tử vong}}{\text{Số ca xác nhận}} * 100\% \quad (2)$$

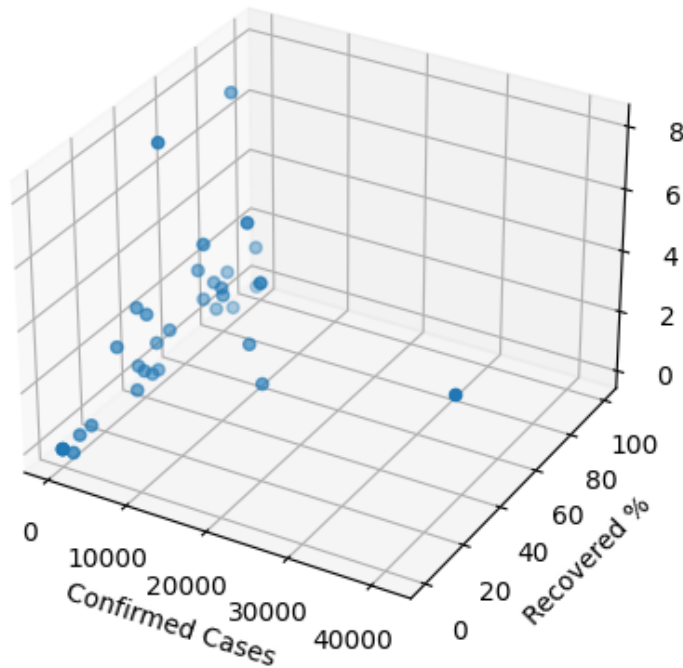
```
1 # Calculate the 'Recovered' and 'Deaths' percentages, ensuring 'Confirmed'
  is not zero
2 df_india['Recovered'] = # Your code here #
3 df_india['Deaths'] = # Your code here #
4 print(df_india)
```

	State	Confirmed	Recovered	Deaths
0	Maharashtra	41642	28.159070	3.491667
1	Tamil Nadu	13967	44.977447	0.680175
2	Gujarat	12910	42.509682	5.987607
3	Delhi	11659	47.748520	1.663951
4	Rajasthan	6227	55.965955	2.424924
5	Madhya Pradesh	5981	47.550577	4.531015

Hình 1: Kết quả sau khi tính tỉ lệ

Bước 3: Trực quan hóa dữ liệu trước khi áp dụng K-Means:

```
1 from mpl_toolkits.mplot3d import Axes3D
2
3 fig = plt.figure()
4 ax = fig.add_subplot(111, projection='3d')
5
6 ax.scatter(df_india['Confirmed'], df_india['Recovered'], df_india['Deaths']
7            ])
8 ax.set_xlabel('Confirmed Cases')
9 ax.set_ylabel('Recovered %')
10 ax.set_zlabel('Deaths %')
11 plt.show()
```

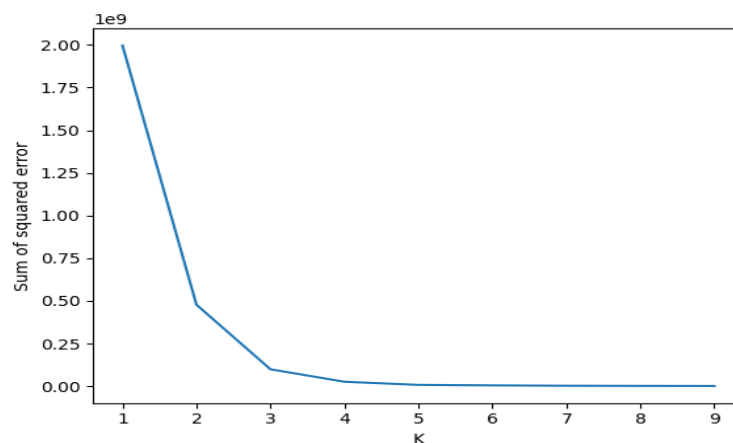


Hình 2: Phân bố data covid trước khi phân cụm

Bước 4: Cài đặt K-Means:

Chúng ta sẽ áp dụng mô hình K-Means và áp dụng phương pháp Elbow để chọn K tốt nhất trong tập dữ liệu này:

```
1 sse = []  
2 k_rng = range(1,10)  
3 # Your code here #
```



Hình 3: Kết quả Elbow - Qua đồ thị này dễ thấy K = 4 là kết quả tối ưu

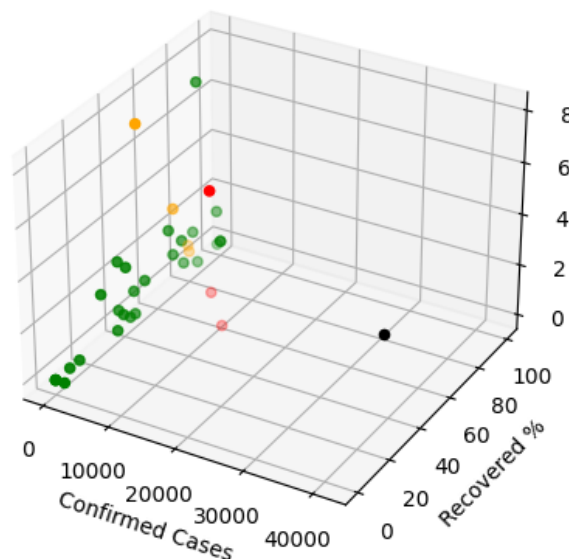
Bước 5: Mapping kết quả cluster vào dữ liệu để trực quan hóa trên bản đồ thế giới:

Chúng ta sẽ mapping kết quả cluster trên với $K = 4$ vào dữ liệu với cột mới có tên là **Zone**, giá trị cột này là giá trị clustering từ mô hình K-Mean với $K = 4$

```
1 # Your code here #
2 print(df_india)
```

	State	Confirmed	Recovered	Death	Zone
0	Maharashtra	41642	28.159070	3.491667	3
1	Tamil Nadu	13967	44.977447	0.680175	2
2	Gujarat	12910	42.509682	5.987607	2
3	Delhi	11659	47.748520	1.663951	2
4	Rajasthan	6227	55.965955	2.424924	1
5	Madhya Pradesh	5981	47.550577	4.531015	1

Hình 4: Kết quả mapping zone



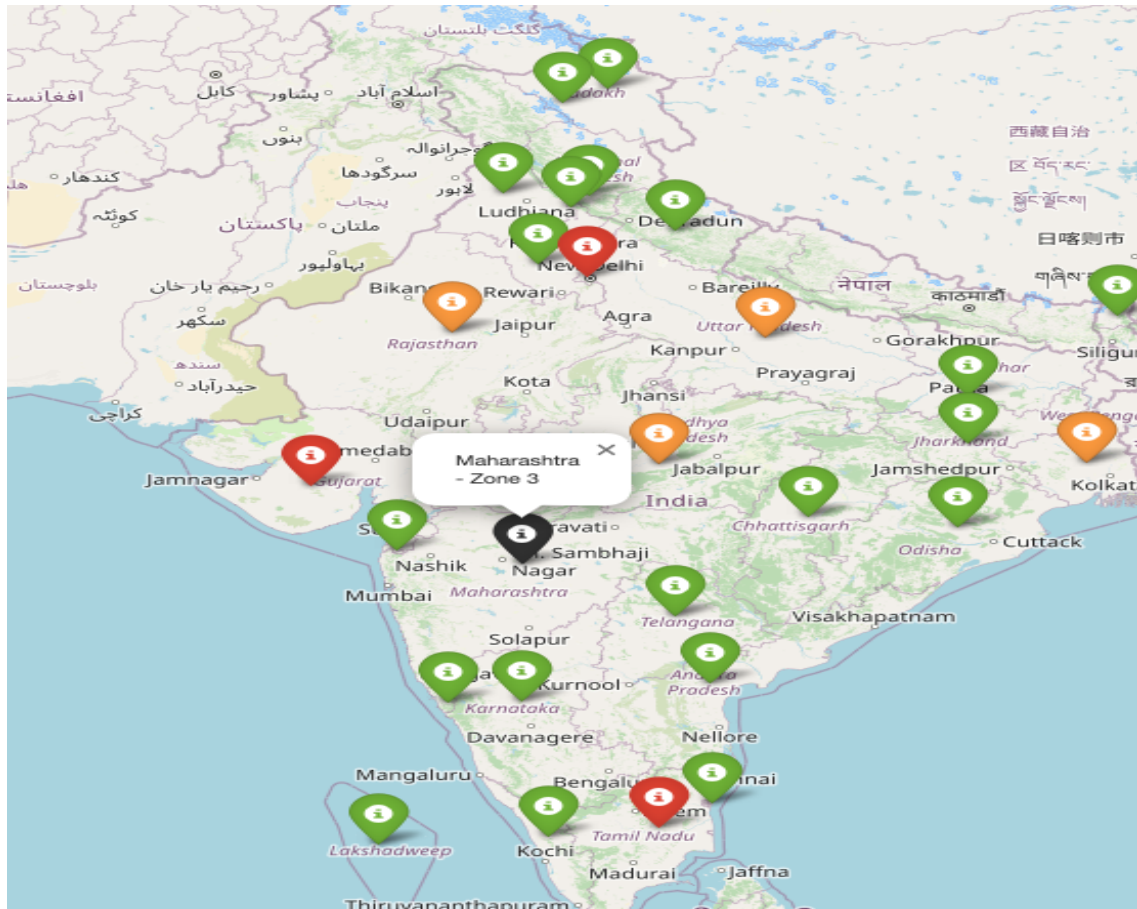
Hình 5: Kết quả phân bố sau khi thực hiện phân cụm

Bước 6: Trực quan hóa phân cụm trên bản đồ thế giới:

Để thực hiện trực quan hóa trên bản đồ thế giới, chúng ta cần cài đặt thêm thư viện Folium để giúp vẽ bản đồ thế giới. Cũng như các có dữ liệu kinh tuyến và vĩ tuyến của các bang của Ấn Độ. Trong project này, chúng ta đã được cung cấp dữ liệu các tọa độ của các bang, tuy nhiên đối với bài toán thực tế các bạn có thể lấy dữ liệu tọa độ theo dữ liệu GeoJSON Tại đây. Chúng ta sẽ thực hiện trực quan hóa toàn bộ phân cụm lên bản đồ thế giới với Folium như sau:

```
1 !pip install folium
2 import folium
3
4 # Assuming you have a dataframe called df_india with 'State' and 'Zone'
  columns
5 # and a dictionary called state_coords with state names as keys and
  latitude, longitude tuples as values.
6
7 # Example state_coords dictionary (replace with your actual data)
8 state_coords = {
9     "Andhra Pradesh": (15.9129, 79.7399),
10    "Arunachal Pradesh": (28.2180, 94.7278),
11    "Assam": (26.2006, 92.9376),
12    "Bihar": (25.0961, 85.3131),
13    "Chhattisgarh": (21.2787, 81.8661),
14    "Goa": (15.2993, 74.1240),
15    "Gujarat": (22.2587, 71.1924),
16    "Haryana": (29.0588, 76.0856),
17    "Himachal Pradesh": (31.1048, 77.1734),
18    "Jharkhand": (23.6102, 85.2799),
19    "Karnataka": (15.3173, 75.7139),
20    "Kerala": (10.8505, 76.2711),
21    "Madhya Pradesh": (22.9734, 78.6569),
22    "Maharashtra": (19.7515, 75.7139),
23    "Manipur": (24.6637, 93.9063),
24    "Meghalaya": (25.4670, 91.3662),
25    "Mizoram": (23.1645, 92.9376),
26    "Nagaland": (26.1584, 94.5624),
27    "Odisha": (20.9517, 85.0985),
28    "Punjab": (31.1471, 75.3412),
29    "Rajasthan": (27.0238, 74.2179),
30    "Sikkim": (27.5330, 88.5122),
31    "Tamil Nadu": (11.1271, 78.6569),
32    "Telangana": (18.1124, 79.0193),
33    "Tripura": (23.9408, 91.9882),
34    "Uttar Pradesh": (26.8467, 80.9462),
35    "Uttarakhand": (30.0668, 79.0193),
36    "West Bengal": (22.9868, 87.8550),
37    "Andaman and Nicobar Islands": (11.7401, 92.6586),
38    "Chandigarh": (30.7333, 76.7794),
39    "Dadra and Nagar Haveli and Daman and Diu": (20.2270, 73.0169),
40    "Delhi": (28.7041, 77.1025),
41    "Jammu and Kashmir": (33.7782, 76.5762),
42    "Ladakh": (34.1526, 77.5806),
```

```
43     "Lakshadweep": (10.5593, 72.6358),
44     "Puducherry": (11.9416, 79.8083),
45     "Orissa": (20.9517, 85.0985) # Added Orissa for consistency
46 }
47
48
49 # Create a map centered on India
50 map_india = folium.Map(location=[20.5937, 78.9629], zoom_start=4)
51
52 # Add markers for each state with color based on cluster
53 for index, row in df_india.iterrows():
54     state = row['State']
55     zone = row['Zone']
56     if state in state_coords:
57         lat, lon = state_coords[state]
58         if zone == 0:
59             color = "green"
60         elif zone == 1:
61             color = "orange"
62         elif zone == 2:
63             color = "red"
64         else:
65             color = "black"
66         folium.Marker(
67             location=[lat, lon],
68             popup=f"{state} - Zone {zone}",
69             icon=folium.Icon(color=color)
70         ).add_to(map_india)
71
72 # Display the map
73 map_india
```



Hình 6: Kết quả phân vùng các mức độ bệnh Covid ở Ấn Độ

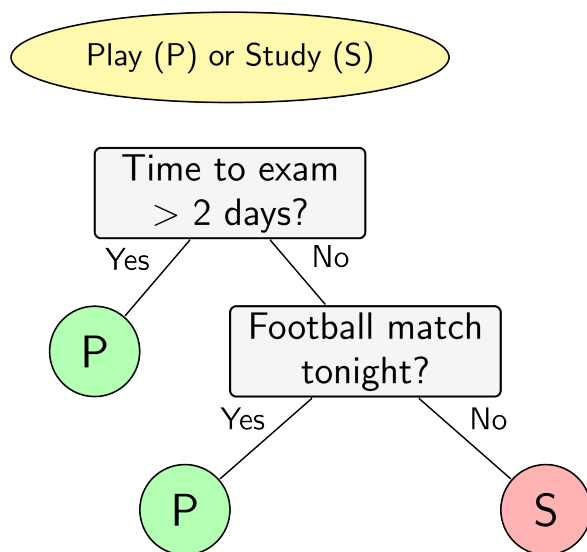
- Hết -

Decision Tree Quizz

Hoàng-Nguyên Vũ

1. Mô tả:

- **Decision Tree** là một trong những thuật toán supervised-learning đơn giản nhất trong Machine Learning. Thuật toán này dựa trên các node được xây dựng từ trước và rẽ nhánh phù hợp để nhằm đưa ra kết quả cho bài toán.



Hình 1: Ví dụ về Decision Tree

2. Bài tập: Lưu ý: Một số câu có trên 2 đáp án

Câu 1. Hãy nêu ra sự khác biệt chính của việc áp dụng Decision Tree vào bài toán Classification và Regression ?

- Thuật toán Decision Tree sử dụng ý tưởng Entropy và GINI cho bài toán Classification và ý tưởng Mean Square Error cho bài toán Regression
- Thuật toán Decision Tree sử dụng ý tưởng Mean Square Error cho bài toán Classification và ý tưởng Entropy cho bài toán Regression
- Thuật toán Decision Tree sử dụng ý tưởng tính khoảng cách Euclidean cho bài toán Classification và ý tưởng tính khoảng cách Mahattan cho bài toán Regression
- Thuật toán Decision Tree sử dụng ý tưởng tính khoảng cách Mahattan cho bài toán Classification và ý tưởng tính khoảng cách Euclidean cho bài toán Regression

- **Đáp Án:** A - Vì ý tưởng chính của giải thuật Decision Tree: Entropy và Gini cho bài toán Classification và ý tưởng Mean Square Error cho bài toán Regression.

Câu 2. Quan sát đoạn code sau:

```
1 # Paragraph B
2 df = pd.read_csv('Salary_Data_simple.csv')
3
4 # Paragraph A
5 import numpy as np
6 import pandas as pd
7 import matplotlib.pyplot as plt
8 from sklearn.tree import DecisionTreeRegressor
9
10 # Paragraph D
11 dt_regressor = DecisionTreeRegressor(max_depth=2)
12 dt_regressor.fit(X, y)
13 y_pred_train = dt_regressor.predict(X)
14 y_pred = dt_regressor.predict(X_test)
15
16 # Paragraph C
17 X = df.iloc[:, -1]
18 y = df.iloc[:, -1]
19 X_train, X_test, y_train, y_test = train_test_split(X, y,
20                                                    random_state = 0)
```

Thứ tự đúng của các đoạn trên là:

- A) A - C - B - D
- B) A - D - B - C
- C) A - B - C - D
- D) A - B - D - C

- **Đáp Án:** C - Vì theo thứ tự khi code: import thư viện → đọc file dataset → Chia dữ liệu train/test → dựng mô hình và kiểm tra trên tập test.

Câu 3. Để giảm tỉ lệ overfitting cho Decision Tree, chúng ta sử dụng kĩ thuật gì ?

- A) Rebuilding Trees
- B) Prunning
- C) Boosting
- D) None of the above

- **Đáp Án:** B - để giảm thiểu overfitting trong Decision Tree, kỹ thuật Prunning sẽ giúp chúng ta chặt bớt nhánh của cây, giúp giải thuật không bị overfit.

Câu 4. Entropy trong Machine Learning là gì ?

- A) Là thuật ngữ đánh giá sự hỗn loạn của các phần tử trong vũ trụ.
- B) Không đáp án chính xác.
- C) Là một thuật ngữ được xài trong Decision Tree bởi cái tên bí ẩn.
- D) Là thuật ngữ đo lường về thông tin đánh giá mức độ chắc chắn của một dataset.

- **Đáp Án:** D.

- Câu 5.** Đây là lý do khiến chúng ta sử dụng hàm logarithm trong khi tính toán Entropy?
- A) Để mô hình phân biệt với cách tính Gini
 - B) Bởi vì hàm logarithm được lập trình trong máy tính dễ dàng.
 - C) Để quy chuẩn thông tin về mặt độ lớn về cùng một tham chiếu.
 - D) Bởi vì nếu không sử dụng thì các con số được xử lý sẽ rất lớn.

- **Đáp Án:** C.

- Câu 6.** Cho biết Big-O Notation, ký hiệu là $O()$ là công cụ đánh giá thời gian chạy của một thuật toán. Ví dụ: Thuật toán cộng các giá trị từ 1 tới n vào một biến sẽ có Big-O Notation là $O(N)$.

Cho biết N là số lượng mẫu cho thuật toán, k là số lượng features, d là độ sâu của cây, hãy tính toán Big-O Notation thuật toán Decision Tree được xây dựng ?

- A) $O(N^{**2}kd)$
- B) $O(Nkd)$
- C) $O(N)$
- D) $O(Nkd^{**2})$

- **Đáp Án:** B - Độ phức tạp thời gian và bộ nhớ của thuật toán Decision Tree phụ thuộc vào:

- + N : Số lượng mẫu trong tập dữ liệu
- + k : Số lượng features (thuộc tính)
- + d : Độ sâu của cây

Giai đoạn dự đoán: Duyệt cây từ gốc đến node lá: $O(d)$ → Tính toán kết quả cho mỗi mẫu: $O(k)$ → Tổng thời gian dự đoán trên toàn tập N mẫu data: $O(n * d * k)$

- Câu 7.** Lý do **chính** khi tính GINI tổng, chúng ta cần nhân thêm hệ số cho mỗi nhánh của node chính ?

- A) Bởi để node chính trong trường hợp này không bị thua thiệt khi so sánh với các trường hợp khác.
- B) Bởi nếu không thì GINI tổng của chúng ta sẽ vượt quá giá trị tối đa có thể.
- C) Bởi để phân biệt sự khác nhau giữa mỗi nhánh
- D) Bởi để đảm bảo sự đóng góp cho mỗi nhánh của node chính.

- **Đáp Án:** D.

- Câu 8.** Tiếp tục quan sát đoạn code dưới đây:

```
1 # Paragraph A
2 def gini_split_a(attribute_name):
3     attribute_values = df1[attribute_name].value_counts()
4     gini_A = 0
5     for key in attribute_values.keys():
6         df_k = df1[class_name][df1[attribute_name] == key].
7         value_counts()
8         n_k = attribute_values[key]
```

```

8         n = df1.shape[0]
9         gini_A = gini_A + ((n_k / n) * gini_impurity(df_k))
10    return gini_A
11
12 gini_attribute = {}
13
14 # Paragraph B
15 def gini_impurity(value_counts):
16     n = value_counts.sum()
17     p_sum = 0
18     for key in value_counts.keys():
19         p_sum = p_sum + (value_counts[key] / n) * (value_counts[
20 key] / n)
21     gini = 1 - p_sum
22     return gini
23
24 class_value_counts = df1[class_name].value_counts()
25 gini_class = gini_impurity(class_value_counts)
26
27 # Paragraph C
28 min_value = min(gini_attribute.values())
29 selected_attribute = min(gini_attribute.keys())

```

Hãy đặt tên tương ứng cho nhiệm vụ ở mỗi đoạn:

A.

Paragraph A: Calculate Gini

Paragraph B:: Calculate Gini Impurity for the attributes

Paragraph C: Compute Gini gain values to find the best split, an attribute has maximum Gini gain is selected for splitting.

B.

Paragraph A:: Calculate Gini Impurity for the attributes.

Paragraph B: Calculate Gini.

Paragraph C: Compute Gini gain values to find the best split, an attribute has maximum Gini gain is selected for splitting.

C.

Paragraph A: Calculate Gini Impurity for the attributes, an attribute has maximum Gini gain is selected for splitting.

Paragraph B: Calculate Gini.

Paragraph C: Compute Gini gain values to find the best split.

D.

Paragraph A: Calculate Gini.

Paragraph B: Calculate Gini Impurity for the attributes, an attribute has maximum Gini gain is selected for splitting.

Paragraph C: Compute Gini gain values to find the best split.

- **Đáp Án:** B.

Câu 9. Các loại Decision Tree phổ biến là gì ?

- A. SVM, KNN, Naive Bayes.
- B. Linear Regression, Logistic Regression, Decision Tree.
- C. Bởi vì thuật toán Decision Tree được xây dựng giống với các ra quyết định của con người hơn.
- D. ID3, C4.5, CART.

- **Đáp Án:** D. ID3, C4.5 và CART đều là các thuật toán cây quyết định, là một loại mô hình học máy sử dụng cấu trúc dạng cây để phân loại hoặc dự đoán điểm dữ liệu. Chúng hoạt động bằng cách chia dữ liệu thành các tập hợp con ngày càng nhỏ hơn dựa trên các tính năng (thuộc tính) nhất định của dữ liệu, cuối cùng đi đến nút lá đại diện cho phân loại hoặc dự đoán.

Câu 10. Cho một tập Dataset như hình dưới đây:

Bạn hãy xây dựng cây quyết định và lần lượt chọn các cột theo thứ tự **Love Art, Love Nature, Love Math, Love Physics** làm node gốc để quyết định tỉ lệ **Love AI**. Sau đó, hãy tính tổng **GINI Impurity** cho từng lựa chọn và xem xét nên lựa chọn thông số nào làm node gốc

- A) Love Art: 0.417, Love Nature: 0.476, Love Math: 0.5, Love Physics: 0.5, Root: Love Art
- B) Love Art: 0.5, Love Nature: 0.5, Love Math: 0.417, Love Physics: 0.476, Root: Love Math
- C) Love Art: 0.5, Love Nature: 0.5, Love Math: 0.476, Love Physics: 0.417, Root: Love Physics
- D) Love Art: 0.476, Love Nature: 0.417, Love Math: 0.5, Love Physics: 0.5, Root: Love Nature
- E) Love Art: 0.416, Love Nature: 0.476, Love Math: 0.5, Love Physics: 0.5, Root: Love Art
- F) Love Art: 0.367, Love Nature: 0.492, Love Math: 0.394, Love Physics: 0.412, Root: Love Art

NO	LOVE ART	LOVE NATURE	LOVE MATH	LOVE PHYSICS	LOVE AI
1	TRUE	FALSE	TRUE	FALSE	FALSE
2	FALSE	FALSE	TRUE	FALSE	TRUE
3	FALSE	FALSE	TRUE	TRUE	TRUE
4	TRUE	TRUE	FALSE	TRUE	TRUE
5	TRUE	FALSE	TRUE	FALSE	FALSE
6	FALSE	TRUE	FALSE	TRUE	FALSE
7	FALSE	FALSE	TRUE	FALSE	TRUE
8	FALSE	TRUE	FALSE	TRUE	FALSE
9	TRUE	FALSE	TRUE	FALSE	FALSE
10	FALSE	FALSE	FALSE	FALSE	TRUE

Hình 2: Dataset cho sẵn

- **Đáp Án:** E. Các bạn có thể xem lại công thức tính Gini: $G = 1 - \sum_{i=1}^c (p_i)^2$.

Câu 11. Cross-validation là gì ?

- A) Kỹ thuật đánh giá hiệu suất của mô hình trên nhiều tập dữ liệu khác nhau.
- B) Kỹ thuật huấn luyện mô hình trên nhiều tập dữ liệu khác nhau.
- C) Kỹ thuật chọn lựa các thuộc tính tốt nhất để xây dựng cây quyết định.
- D) Tất cả đáp án trên.

- **Đáp Án:** A - Cross-validation là một kỹ thuật được sử dụng để đánh giá hiệu suất của mô hình học máy trên nhiều tập dữ liệu khác nhau. Kỹ thuật này giúp giảm thiểu sai số và tăng độ tin cậy của kết quả đánh giá.

Câu 12. Chúng ta đã biết Gini và Entropy và hay cách để xây dựng Decision Tree cho bài toán Classification. Vậy đâu là lý do lý giải cho việc Gini được sử dụng thường xuyên hơn trong các bài toán thực tế?

- A) Do Entropy là một khái niệm phức tạp và khó hiểu hơn.
- B) Do thuật toán sử dụng Entropy có thời gian tính toán chậm hơn (bởi việc sử dụng hàm logarithm).
- C) Do trong bài báo tác giả đã thử nghiệm trên rất nhiều trường hợp, và thực tế cho thấy rằng việc sử dụng Entropy lại bất ngờ cho kết quả thấp hơn.
- D) Do Gini được khám phá gần đây hơn. Cùng với sự bùng nổ của Machine Learning gần đây thì Gini cũng được ưu chuộng hơn. (1912 so với 1850)

- **Đáp Án:** B.

Câu 13. Đâu là lý do **chính** mà chúng ta không chia Decision Tree tới Gini bằng 0 ?

- A) Thuật toán sẽ chạy rất lâu

- B) Điều này sẽ khiến thuật toán được sinh ra có tỉ lệ overfitting rất cao.
- C) Tốn nhiều thời gian chia cây ở các tập dữ liệu lớn.
- D) Chúng ta không thể đạt được trường hợp có GINI bằng 0.

- **Đáp Án:** B.

Câu 14. Pruning là gì?

- A. Kỹ thuật tăng kích thước của cây quyết định để cải thiện độ chính xác.
- B. Kỹ thuật chọn lựa các thuộc tính tốt nhất để xây dựng cây quyết định.
- C. Kỹ thuật cắt tỉa các nhánh của cây quyết định để giảm thiểu overfitting.
- D. Tất cả đáp án trên.

- **Đáp Án:** C.

Câu 15. Đâu là lời giải thích xác đáng cho 2 khái niệm Bias và Variance ?

- A. Bias là thông số đánh giá độ lỗi trong quá trình training, Variance là thông số đánh giá độ chênh lệch giữa lỗi trong quá trình training và testing.
- B. Bias là thông số đánh giá độ lỗi trong quá trình testing, Variance là thông số đánh giá độ chênh lệch giữa lỗi trong quá trình training và testing.
- C. Variance là thông số đánh giá độ lỗi trong quá trình training, Bias là thông số đánh giá độ chênh lệch giữa lỗi trong quá trình training và testing.
- D. Variance là thông số đánh giá độ lỗi trong quá trình testing, Bias là thông số đánh giá độ chênh lệch giữa lỗi trong quá trình training và testing.

- **Đáp Án:** A và B.

(*) **Ôn tập Toán Xác Suất cơ bản**

Câu 16. Gieo một con xúc xắc 6 mặt cân đối 2 lần. Xác suất để tổng số chấm xuất hiện trong hai lần gieo là 7 là ?

- A. $1/36$
- B. $1/6$
- C. $1/12$
- D. $1/18$

- **Đáp Án:** B.

1. Xác định số kết quả có thể xảy ra: Khi gieo hai con xúc xắc 6 mặt cân đối, mỗi con có 6 khả năng xuất hiện (từ 1 đến 6). Do đó, có $6 * 6 = 36$ kết quả có thể xảy ra.
2. Xác định số kết quả thuận lợi: Để tổng số chấm xuất hiện trong hai lần gieo là 7, có 6 trường hợp sau: (1, 6), (2, 5), (3, 4), (4, 3), (5, 2), (6, 1)
3. Xác suất để tổng số chấm xuất hiện trong hai lần gieo là 7 là: $6/36 = 1/6$

Câu 17. Ba người cùng bắn vào một bia. Xác suất để người thứ nhất, thứ hai, thứ ba bắn trúng đích lần lượt là 0,8; 0,6; 0,5. Xác suất để có đúng 2 người bắn trúng đích là ?

- A. 0.24
- B. 0.96
- C. 0.46
- D. 0.92

- **Đáp Án:** C. Gọi ba người cùng bắn vào 1 bia với xác suất 0,8; 0,6; 0,5 lần lượt là A, B, C.

+ TH1: A, B bắn trúng, C không bắn trúng nên xác suất $P_1 = P_A * P_B * (1 - P_C) = 0.24$

+ TH2: A, C bắn trúng, B không bắn trúng nên xác suất $P_2 = P_A * (1 - P_B) * P_C = 0.16$

+ TH3: C, B bắn trúng, A không bắn trúng nên xác suất $P_3 = (1 - P_A) * P_B * P_C = 0.06$

Vậy xác suất cần tính là tổng xác suất 3 TH trên: 0.46

Câu 18. Một lô hàng có 100 sản phẩm, biết rằng trong đó có 8 sản phẩm hỏng. Người kiểm định lấy ra ngẫu nhiên từ đó 5 sản phẩm. Tính xác suất của biến cố A: “Người đó lấy được đúng 2 sản phẩm hỏng” ?

- A. 0.046
- B. 0.084
- C. 0.146
- D. 0.208

- **Đáp Án:** A. Số phần tử của không gian mẫu: $\omega = C_{100}^5$. Trong 100 sản phẩm đó có 8 sản phẩm hỏng và 92 sản phẩm không hỏng nên số phần tử của biến cố A là: $n(A) = C_8^2 * C_{92}^3$. Vậy xác suất như đề bài sẽ là $\frac{n(A)}{\omega} = 0.046$

Câu 19. Một hộp đựng 10 viên bi trong đó có 4 viên bi đỏ, 3 viên bi xanh, 2 viên bi vàng, 1 viên bi trắng. Lấy ngẫu nhiên 2 bi tính xác suất biến cố : A: “2 viên bi cùng màu” ?

- A. 1/9
- B. 2/9
- C. 1/3
- D. 4/9

- **Đáp Án:** A. Số phần tử của không gian mẫu: $\omega = C_{10}^2$. Gọi các biến cố: D: “lấy được 2 viên đỏ” ; X: “lấy được 2 viên xanh” ; V: “lấy được 2 viên vàng”. Ta có D, X, V là các biến cố đôi một xung khắc và $C = D \cup X \cup V$. Vậy $P(C) = P(D) + P(X) + P(V) = \frac{C_4^2}{C_{10}^2} + \frac{C_3^2}{C_{10}^2} + \frac{C_2^2}{C_{10}^2} = \frac{2}{9}$

- Hết -